

Why Discovery Needs a Judge

Surrogate Judges for Autonomous Hypothesis Generation

MoLinG-666

September 24, 2026 · Position paper · Draft v0.1

The experiments described here have not yet been conducted. All numbers and figures are illustrative.

Abstract

The bottleneck of autonomous scientific discovery is not generating hypotheses but verifying them. Purely language-based generation and multi-agent critique lack independent external feedback and are prone to what we call *hallucination resonance*. Real experiments provide genuine feedback but are too slow and too expensive to absorb massive trial and error. We therefore propose inserting a **surrogate judge** between the generator and reality to filter out clearly invalid candidates first.

Our central claim is that **the best judge is not necessarily the most accurate one, but the one whose errors are least correlated with the generator's, at an acceptable verification cost**. We give a measurable definition of resonance strength, a cost model for when an imperfect judge is worth using, and a minimal experiment that runs on an ordinary computer. We also state openly the largest unsolved problem of this approach: where the judge itself comes from, and how far it can be trusted.

1 The Problem: Generation Is Cheap, Verification Is the Bottleneck

Today's large models can write hundreds of "new hypotheses" in seconds. Generation is no longer scarce; what is scarce is knowing which of them are true.

A hypothesis becomes knowledge only after passing some test that is capable of saying "no": an experiment, an observation, a mathematical proof, or a rebuttal from peers. Without that step, any number of ideas remain mere candidates. The question this paper asks is therefore: **when AI generates hypotheses autonomously and at scale, who acts as the judge?** We argue that the quality of the judge, rather than the creativity of the generator, determines how far the whole system can go.

2 Why Self-Critique Is Not Enough

A popular approach is to have several models propose hypotheses and critique one another. The problem is not that they "use language": modern models can call tools, retrieve documents, and process images. The problem is that **the errors of the generator and the judge are highly correlated**. Models taking part in mutual critique are usually trained on similar data and share similar blind spots. As long as the evaluation signal comes mainly from these shared priors, without independent external feedback, self-critique cannot be relied upon to

correct shared mistakes.

Let E_G denote the event “the generator proposes a wrong hypothesis” and E_J the event “the judge accepts a wrong hypothesis”. We define the resonance strength as

$$R_{\text{res}} = P(E_J \mid E_G) - P(E_J). \quad (1)$$

$R_{\text{res}} > 0$ means the judge is more likely than usual to err exactly when the generator errs; we call this **hallucination resonance**. $R_{\text{res}} \approx 0$ means the two sets of errors are approximately independent.

An overall $R_{\text{res}} \approx 0$ does not rule out resonance: one class of critical errors may be strongly correlated while another is negatively correlated, and the two cancel out. In practice, R_{res} should therefore be measured separately for each error type k (for example, violations of conservation laws, dimensional errors, or boundary-condition errors), giving $R_{\text{res}}^{(k)}$.

The definition has a direct corollary: **a judge’s accuracy on a general benchmark overestimates its performance on the generator’s errors**. A judge that scores 95% on a standard test may do much worse on exactly the kinds of mistakes the generator tends to make. Judges must be evaluated on the generator’s actual outputs.

3 Why Not Verify Directly in Reality?

Reality is the most reliable judge, but it is costly. A single wet-lab experiment may take days or months, while a generator can produce thousands of candidates per minute. Reagents, equipment, and labor make most candidates not worth testing, and in fields such as chemistry, biology, and hardware, failed trials can be wasteful or even dangerous. The large speed gap between generation and verification means we need an intermediate layer that reduces the number of candidates first.

4 Proposal: A Surrogate Judge

We propose inserting a **surrogate judge** between the generator and reality: an approximate model that cheaply predicts the consequences of a candidate hypothesis and reports its confidence. We deliberately avoid the term “world model”, which now covers video generators, reinforcement-learning dynamics models, latent predictors, and much more; this paper needs only a narrow instance of the idea. The overall structure is shown in Figure 1.

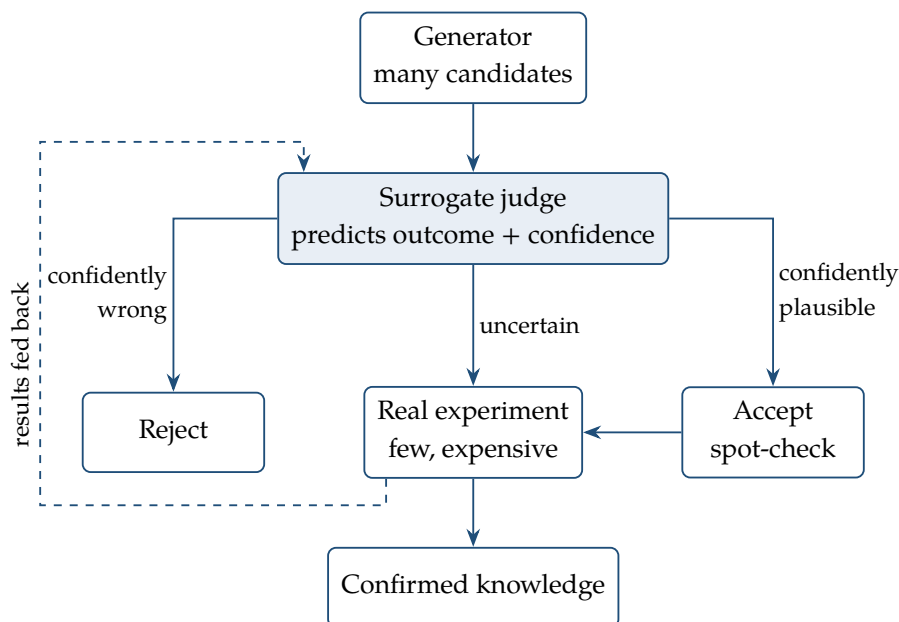


Figure 1. A three-way verification funnel. The surrogate judge rejects a candidate only when confident, and routes uncertain candidates to real experiments.

The surrogate judge need not be perfect, but it must be much faster than reality, trustworthy within the range where it is competent, and **aware of when it cannot be trusted**. Its job is not to confirm truth, but to say “no” cheaply, and to say “I don’t know” honestly when unsure. This is the point of the three-way routing: uncertain candidates are escalated to real experiments instead of being rejected, so that genuinely new discoveries that conflict with old knowledge are not killed by mistake. Results from real experiments must also be fed back to correct the judge; otherwise the system will keep exploiting the judge’s errors.

4.1 Precedents and Related Work

The pattern “loose generation, strict judgment” has well-known precedents. In AlphaGo [1], the rules of Go are themselves a perfect judge; AlphaEvolve [2] scores candidate algorithms with fixed automatic evaluators; biological evolution lets the environment eliminate random variations. All of these, however, are closed domains with **exact or near-exact** judges. They show how powerful a cheap and reliable judge can be, but they are not evidence that our proposal works. The difficulty of open-ended science is that the judge can only approximate the truth.

In open-ended settings, The AI Scientist [3] and the AI co-scientist [4] already implement closed loops of generating, reviewing, and iterating on hypotheses. Closest to this paper are Large Discovery Models [5], which pair a generative model with a Bayesian surrogate that scores candidates, quantifies uncertainty, and is continually updated with real experimental results. Our difference from these works lies not in *whether* to use a judge, but in treating **the correlation between the judge’s and the generator’s errors**, the cost of false rejections, and the verification cost as the core variables for evaluating a judge.

5 A Minimal Testable Experiment: Pendulums and Springs

To make our claim falsifiable, we design a minimal experiment that asks: **can a surrogate simulator learned purely from data reliably filter out wrong hypotheses?**

1. **Data.** Generate trajectories (position and velocity over time) of a pendulum and a spring oscillator from the true physical equations, with a small amount of noise. The true equations, denoted T^* , are used only for final scoring; neither the simulator nor the generator ever sees them.
2. **Train the simulator.** Train a neural network on trajectory data alone to predict the next state, without any formulas. Denote the learned simulator by $\hat{T}$.
3. **Generate candidates.** Have a language model or a random program propose a set of candidate equations of motion, some correct and some violating energy conservation or dimensional consistency.
4. **Filter.** Generate a trajectory from each candidate equation, compare it with the predictions of $\hat{T}$, and reject candidates with large deviations.
5. **Score against the truth.** Use T^* to check the filtering results and compute the false-acceptance rate f_p (fraction of wrong equations accepted) and the false-rejection rate f_n (fraction of correct equations rejected).

Step 4 only shows that a candidate agrees with $\hat{T}$, not that it is correct. It is step 5 that separates “agreeing with the simulator” from “agreeing with the truth”, and thereby avoids circular reasoning.

5.1 Core Comparison: Degree of Judge Independence

To test directly whether “more independent judges are better”, we fix the generator and swap in four judges of increasing independence, with the true physics as an upper-bound reference (Table 1).

Judge	Relation to the generator
Generator self-evaluation	The same model
Another LLM of the same family	Similar training data and architecture
An LLM of a different family	Partially independent training pipeline and architecture, but likely sharing much public data and many biases
Learned surrogate simulator $\hat{T}$	Learns from trajectory data only; reads no text
Upper-bound reference: true physics T^*	Not a judge; used only for final scoring

Table 1. Judges of increasing independence.

For each judge we measure $R_{\text{res}}^{(k)}$, f_p , and f_n . If judges with lower R_{res} perform better overall, even when their general accuracy is not the highest, this supports our central claim; if not, error correlation is not the key variable after all. Figure 2 shows what we expect.

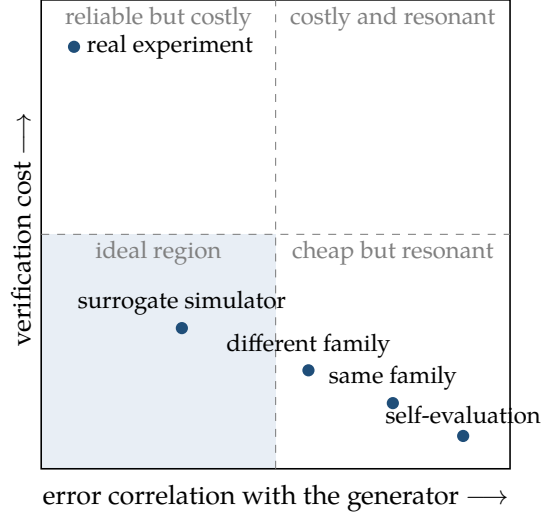


Figure 2. Illustration, not experimental results. Point positions only express our expectations and must be confirmed or refuted by experiment.

The success criterion is that the surrogate simulator filters clearly better than language-only scoring. The more interesting test lies outside the training distribution (larger swing angles, stiffer springs): if the simulator quickly breaks down there, that would confirm the concerns of Section 7. The whole experiment can be run on an ordinary computer.

6 Cost Analysis: When Is an Imperfect Judge Worth Using?

Let N be the number of candidates, of which a fraction p are true; let C_r be the cost of one real experiment and C_s the cost of one simulation; and let f_p and f_n be the judge’s false-acceptance and false-rejection rates. Without a judge, every candidate is tested in reality:

$$\text{Cost}_0 = NC_r. \quad (2)$$

With a judge, only candidates that pass the filter are tested, and the pass rate is $q = p(1 - f_n) + (1 - p)f_p$:

$$\text{Cost}_1 = NC_s + NqC_r. \quad (3)$$

Scientifically, we must also count true discoveries that are rejected by mistake. Let C_{miss} be the cost of missing one true discovery. The total cost is

$$J = NC_s + NqC_r + Npf_nC_{\text{miss}}. \quad (4)$$

The condition $J < \text{Cost}_0$ is equivalent to

$$\frac{C_s}{C_r} + \frac{pf_nC_{\text{miss}}}{C_r} < 1 - q. \quad (5)$$

The left-hand side is the relative cost of simulation plus the relative cost of false rejections; the right-hand side is the fraction of candidates the judge filters out.

An illustrative example (hypothetical numbers): $p = 1\%$, $f_p = 10\%$, $f_n = 20\%$, $C_s/C_r = 0.001$. The judge then filters out about 89% of candidates, saving nearly 90% of the experimental cost. But if missing one true discovery costs 1000 times as much as one experiment, the relative cost of false rejections is about 2, which exceeds 0.89, and the judge is no longer worth using. This is a quantitative version of “the stricter the judge, the more likely it blocks a revolution”: the rarer and more valuable true hypotheses are, the lower f_n must be.

Three-way routing changes this trade-off. Suppose the judge reports “uncertain” for a fraction u of candidates and sends them directly to real experiments. These candidates cannot be falsely rejected, at the price of Nu extra experiments. As long as the judge’s false rejections are concentrated in the region where it is itself unsure, this can greatly reduce f_n at only a modest increase in experimental cost.

Note that f_p and f_n must be measured on the generator’s actual outputs, so the error correlation of Section 2 is already contained in these two numbers. Why, then, define R_{res} separately? Because the two play different roles: f_p and f_n are **operational metrics** that do the accounting and decide whether a judge is worth using, while R_{res} is a **diagnostic metric** that explains why a judge with high general accuracy fails on a particular generator, and guides the choice of a better judge.

The model has obvious simplifications: C_{miss} is hard to estimate in practice; errors are assumed independent across candidates; and it ignores the gradual decrease of f_p and f_n as real results are fed back. The experiment in Section 5 can measure f_p and f_n directly, moving this model from paper to data.

7 Open Problems and Limitations

The greatest weakness of this approach lies not in the generator but in the judge itself. We currently have no answers to the following questions:

- **Where does the judge come from?** A judge learned from data inherits the limits of that data and can only tell whether something “fits what is known”. Hand-written physical rules amount to injecting existing human knowledge, which is impossible in fields without established laws. A hybrid may be the way forward, but where to draw the line remains unclear.
- **How trustworthy is the surrogate?** A judge may be accurate within its training range yet fail silently outside it. We need ways for it to know when it cannot be trusted and to hand those cases over to real experiments.
- **Reward hacking.** If the generator is optimized to “pass the judge”, it may learn to exploit the judge’s flaws, producing hypotheses that hold in simulation but fail in reality.
- **Genuinely new discoveries may violate the judge.** A revolutionary hypothesis looks wrong under the old model. Three-way routing mitigates this, but only if the judge actually reports “uncertain” when facing new phenomena rather than being confidently wrong, which itself needs to be tested.

The stricter the judge, the better it blocks errors, and the more likely it blocks revolutions.

How to balance the two may be the central difficulty of this research direction.

8 Conclusion

What autonomous scientific discovery needs is not more ideas, but a judge that can say “no” and can also say “I don’t know”. The key measure of a judge is not its general accuracy but how independent its errors are from the generator’s. Language-only self-critique, lacking independent external feedback, tends to lack such a judge, while reality is too slow and too expensive; a surrogate judge as an intermediate layer is therefore worth testing seriously.

We do not claim that this approach already works. What we offer is a testable claim, a minimal experiment that can be run on an ordinary computer, a cost model for when a judge pays off, and a set of questions that must be answered head-on. The hardest of them is how to build a judge that blocks errors without also blocking genuinely new discoveries.

References

- [1] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.
- [2] A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.
- [3] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv:2408.06292, 2024.
- [4] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu et al. Towards an AI co-scientist. arXiv:2502.18864, 2025.
- [5] Z. Yu et al. Large Discovery Models: Empirically-grounded model-based open-ended search. arXiv:2608.15669, 2026.